

Tarea 1

INF3841 – Recuperación de Información

Magíster en Inteligencia Artificial

Pontificia Universidad Católica de Chile

Profesor: Juan Manuel Barrios

Fecha: 10 de agosto de 2026

El objetivo de esta tarea es implementar un buscador de imágenes duplicadas o derivadas. Dado un conjunto de imágenes originales R y un conjunto de imágenes derivadas Q , se desea determinar para cada imagen $q \in Q$ si existe alguna imagen $r \in R$ tal que $T(r) = q$ para alguna transformación T .

En la Figura 1 se muestran las transformaciones que usarán para esta tarea: ajustar colores (GAMMA), recortar una zona (CROP), enmarcar (FRAMED) y reflejado horizontal (FLIP). Las imágenes originales fueron obtenidas de `train2014.zip` del COCO dataset.



Figura 1: Imágenes originales y derivadas para las 4 transformaciones a evaluar.

Para la tarea deberá implementar dos comandos: uno para la fase offline y uno para la fase online.

Fase offline del sistema (parte 1)

Este comando recibe el nombre de una carpeta donde están las imágenes originales R y el nombre de una carpeta nueva donde se guardarán los descriptores calculados:

```
python tarea1-parte1.py dir_imagenes_R dir_descriptores_R
```

Para cada imagen en `dir_imagenes_R` calcula uno o más descriptores de contenido (a su elección). Al finalizar guarda en la carpeta `dir_descriptores_R` uno o más archivos con los nombres de las imágenes procesadas y sus descriptores.

Fase online del sistema (parte 2)

Este comando recibe el nombre de la carpeta donde están las imágenes derivadas Q , la carpeta con los descriptores de R (creada por la fase offline) y un nombre del archivo de salida a crear con los resultados:

```
python tarea1-parte2.py dir_imagenes_Q dir_descriptores_R resultados.txt
```

Para cada imagen de Q calcula uno o más descriptores de contenido (a su elección, puede ser distinto al usado en la parte 1 para R). Luego, para cada descriptor de Q busca el descriptor más cercano de R . Finalmente, genera un archivo de texto con el resultado de las búsquedas siguiendo el formato descrito en la siguiente sección.

Archivo de resultados

El archivo de texto a generar por la parte 2 debe tener una fila por imagen en Q y en cada fila tres columnas separadas por tabuladores: nombre de imagen de Q , nombre de imagen de R más parecida y la distancia entre ambas:

```
nombre_imagen_q \t nombre_imagen_r \t distancia_entre_q_r
```

Notar que mientras más parecidas sean las imágenes menor debe ser la distancia entre ellas.

q0001.jpg	r0432.jpg	610.2
q0002.jpg	r0231.jpg	126.1
.....		
q4346.jpg	r0156.jpg	11.5
q4347.jpg	r1849.jpg	119.5

Figura 3: Ejemplo de archivo de resultados

Datos de prueba

Junto con este enunciado se publican tres conjuntos de prueba (llamados `dataset_a`, `dataset_b`, `dataset_c`). Cada conjunto de prueba contiene imágenes originales (conjunto R), imágenes transformadas (conjunto Q) y un archivo `gt.txt` que señala para cada imagen de R su imagen en Q correspondiente.

Programa de Evaluación

También junto con este enunciado encontrará un programa de evaluación que le servirá para probar su tarea sobre los distintos datasets. Para usar el programa de evaluación debe invocar el programa:

```
python evaluarTarea1.py
```

Este programa llama a `tarea1-parte1.py` y `tarea1-parte2.py` con cada dataset de prueba, lee los resultados generados, compara las respuestas con las respuestas correctas en cada dataset, determina la distancia umbral de corte que logra un mejor balance entre respuestas correctas e incorrectas, y finalmente dependiendo de la cantidad de detecciones correctas reporta la nota obtenida.

Su tarea será evaluada en los conjuntos de imágenes publicados y en otros conjuntos similares. Su tarea **no puede demorar más de 10 minutos** en procesar cada conjunto de prueba.

Existe la posibilidad de obtener hasta **1 punto extra de bonus** para otras tareas si logra detectar la mayoría de los duplicados en los datasets de evaluación.

Implementación

La tarea la debe implementar en Python 3 usando cualquier función de OpenCV, NumPy, SciPy y otras librerías gratuitas (exceptuando librerías de machine learning).

Junto con este enunciado encontrará una implementación base para `tarea1-parte.py` y `tarea1-parte2.py`. Se espera que para resolver la tarea use el código fuente de ejemplo publicado en la **sección Archivos de Canvas**.

Reglas de Entrega

- El plazo final de entrega es el **domingo 30 de agosto de 2026 a las 23:59**. Debe subir su respuesta en la sección Tareas de Canvas. No se aceptan atrasos. No se revisarán entregas por correo ni entregas en los comentarios.
- Debe entregar sólo el código fuente de su tarea (archivos .py). No se permiten archivos .ipynb ni links para descargar. No envíe datasets ni descriptores ya calculados.
- Se recomienda además incluir un archivo de texto señalando el sistema operativo en que realizó su tarea e incluir la salida que obtuvo al correr `evaluarTarea1.py` en su computador.
- La tarea debe funcionar y cumplir el enunciado para poder ser evaluada. No se podrán evaluar las tareas incompletas, que arrojen error o que no cumplan el enunciado.
- La tarea es **individual** y debe ser de **su autoría**, es decir, no pueden ser resueltos por otro estudiante, no puede copiar respuestas de Internet, **no se permite usar ChatGPT** ni similares. En caso de detectar copia o plagio se asignará nota 1.0 a las o los estudiantes involucrados.